CLASE 31

Django Models

De clases Python a tablas SQL, sin escribir SQL

- ¿Qué es un ORM y por qué usarlo?
- Definir modelos con clases Python
- Tipos de campos (CharField, IntegerField, etc.)
- Migraciones: makemigrations y migrate
- Registrar modelos en el Admin
- Crear datos desde el Admin

Crear los modelos de Categoría y Producto

¿Qué es un ORM?

ORM = Object-Relational Mapping | Traduce entre objetos Python y tablas SQL

Sin ORM (SQL manual)

```
cursor.execute('''
  CREATE TABLE productos (
    id INTEGER PRIMARY KEY,
    nombre VARCHAR(100),
    precio DECIMAL(10,2)
  )
''')
```

Con ORM (Django)

```
class Producto(models.Model):
    nombre = CharField(100)
    precio = DecimalField()

Producto.objects.create(
    nombre='Laptop', precio=999
)
```

Ventajas del ORM:

- Escribes Python, no SQL
- Funciona con cualquier base de datos
- Previene SQL injection automáticamente
- Más legible y mantenible

Clase Python = Tabla SQL
Objeto = Fila | Atributo = Columna

Anatomía de un modelo Django

Un modelo es una clase que hereda de models.Model

```
class NombreModelo(models.Model):
    campo1 = models.TipoCampo(...)
    campo2 = models.TipoCampo(...)

    class Meta:
        ordering = ['campo1']

    def __str__(self):
        return self.campo1
```

← Hereda de Mode

← Atributo = Colum

← Configuración

← Representación

Traducción automática:

Clase Python → Tabla SQL

Atributo → Columna

Instancia → Fila/Registro

TipoCampo → Tipo de dato SQL

Django agrega automáticamente un campo 'id' como Primary Key (autoincremental)

Modelo: Categoría

Empecemos simple — una categoría de productos

Campos necesarios:

nombre → Texto corto (ej: "Electrónica")

descripcion → Texto largo (opcional)

activa → Booleano (¿mostrar en la tienda?)

SQL equivalente:

```
CREATE TABLE productos_categoria (
  id INTEGER PRIMARY KEY,
  nombre VARCHAR(100) NOT NULL,
  descripcion TEXT,
  activa BOOLEAN DEFAULT TRUE
);
```

Convención de nombres:

Modelo: Singular, PascalCase → Categoria

Tabla: app_modelo en minúsculas → productos_categoria

Creemos el modelo juntos — abre VS Code en productos/models.py

Modelo: Producto

Un producto más completo con relación a Categoría

Campos necesarios:

nombre → Texto corto
descripcion → Texto largo
precio → Número decimal
stock → Número entero
categoria → Relación con Categoría (FK)
imagen_url → URL de imagen (opcional)
activo → Booleano
creado → Fecha automática
actualizado → Fecha automática

Relación con Categoría:

Una categoría tiene muchos productos
Un producto pertenece a una categoría

on_delete:

CASCADE → Borrar producto si se borra categoría
PROTECT → No permitir borrar categoría

ForeignKey = Relación uno a muchos | Django crea la columna categoria_id automáticamente

Tipos de campos más usados

Texto

```
CharField(max_length=N)
TextField()
EmailField()
URLField()
SlugField()
```

Números

```
IntegerField()
DecimalField(max_digits, decimal_places)
FloatField() — no para $$
PositiveIntegerField()
```

Fecha/Hora

```
DateField()
TimeField()
DateTimeField()
```

Otros

```
BooleanField()
FileField()
ImageField()
JSONField()
```

Para precios SIEMPRE DecimalField — Float puede tener errores de precisión

Opciones de campos

Opciones que aplican a casi todos los campos

`null=True`	Permite NULL en la BD
`blank=True`	Permite vacío en formularios
`default=valor`	Valor por defecto
`unique=True`	Valor único (no repetido)
`choices=[...]`	Lista de opciones permitidas
`verbose_name`	Nombre legible para el Admin
`help_text`	Texto de ayuda en formularios

null vs blank:

null = Base de datos
¿Puede ser NULL en la tabla?

blank = Validación
¿Puede estar vacío en un form?

Strings: solo blank=True
Números/fechas: ambos

Regla: null=True para no-strings | blank=True para strings opcionales

Relaciones entre modelos

ForeignKey (1 a N)

Una categoría → muchos productos

```
categoria = models.ForeignKey(Categoria, ...)
```

ManyToMany (N a N)

Un producto → muchas etiquetas y viceversa

```
etiquetas = models.ManyToManyField(Etiqueta)
```

OneToOne (1 a 1)

Un usuario → un perfil

```
usuario = models.OneToOneField(User, ...)
```

on_delete:

CASCADE	→ Borrar también
PROTECT	→ No permitir borrar
SET_NULL	→ Poner NULL
SET_DEFAULT	→ Poner valor default

Clase 32: Relaciones en profundidad — hoy solo usamos ForeignKey

Migraciones — Control de versiones para la BD

Un archivo que describe cambios en la base de datos (crear tablas, agregar columnas, modificar tipos...)

1. Modificas models.py
2. makemigrations (crea el plano)
3. migrate (construye)
4. Cambios en la BD

Analogía: makemigrations = Dibujar el plano de una casa | migrate = Construir según el plano

Las migraciones se guardan en app/migrations/ — Son archivos Python versionables con Git

NUNCA modifiques la BD directamente. Siempre usa migraciones.

Crear y aplicar migraciones

Paso 1 — Crear la migración

```
python manage.py makemigrations
```

→ Detecta cambios en models.py
→ Crea archivo en app/migrations/
→ El archivo describe los cambios

Paso 2 — Aplicar la migración

```
python manage.py migrate
```

→ Ejecuta migraciones pendientes
→ Modifica la base de datos
→ Registra qué ya se aplicó

Comandos útiles:

`makemigrations nombre_app`	→ Solo para esa app
`migrate nombre_app`	→ Solo esa app
`showmigrations`	→ Ver estado de migraciones
`sqlmigrate app 0001`	→ Ver el SQL generado

BREAK — 10 MINUTOS

Hasta ahora:

- Modelo Categoría creado
- Modelo Producto creado
- Migraciones aplicadas

Volvemos con:

- Registrar en el Admin
- Crear datos de prueba
- Usar modelos en views

Admin de Django — Registrar modelos

El Admin viene incluido pero hay que registrar los modelos

```
productos/admin.py
from django.contrib import admin
from .models import Categoria, Producto

admin.site.register(Categoria)
admin.site.register(Producto)
```

Con esto ya puedes:

- Ver lista de registros
- Crear nuevos registros
- Editar registros existentes
- Eliminar registros
- Buscar y filtrar

Acceso: http://127.0.0.1:8000/admin/

__str__ en el modelo define cómo se muestra cada registro en el Admin

Personalizar el Admin

El registro básico funciona, pero podemos mejorarlo mucho

```
@admin.register(Producto)
class ProductoAdmin(admin.ModelAdmin):
    list_display = ['nombre', 'precio',
                    'stock', 'activo']
    list_filter = ['categoria', 'activo']
    search_fields = ['nombre', 'descripcion']
    list_editable = ['precio', 'stock']
```

Opciones de personalización:

`list_display`	Columnas en la lista
`list_filter`	Filtros en el sidebar
`search_fields`	Campos para buscar
`list_editable`	Editar en la lista
`ordering`	Orden por defecto
`readonly_fields`	Solo lectura
`date_hierarchy`	Navegación por fecha

Un buen Admin ahorra MUCHO tiempo de desarrollo

Django Shell — Interactuar con modelos

```
python manage.py shell
```

Obtener todos

```
Producto.objects.all()
```

Obtener uno por id

```
Producto.objects.get(id=1)
```

Filtrar

```
Producto.objects.filter(activo=True)
```

Filtrar por precio

```
Producto.objects.filter(precio__lte=100)
```

Crear

```
Producto.objects.create(nombre='Test', ...)
```

Contar

```
Producto.objects.count()
```

El shell es perfecto para experimentar con queries antes de usarlos en views

Conectar modelos con views

Antes (datos fake)

```
PRODUCTOS = [
  {'id': 1, 'nombre': 'Laptop'},
]
def lista(request):
  return render(request,
    'lista.html', {'productos': PRODUCTOS})
```

Ahora (datos reales)

```
from .models import Producto

def lista(request):
  productos = Producto.objects
    .filter(activo=True)
  return render(request,
    'lista.html', {'productos': productos})
```

El template no cambia:

```
{% for producto in productos %}  {{ producto.nombre }} - ${{ producto.precio }}  {% endfor %}
```

Solo cambia de dónde vienen los datos

EJERCICIO: Crear modelo Venta

Para el Dashboard necesitamos registrar ventas

Crear modelo Venta con:

producto → FK a Producto

cantidad → IntegerField

precio_unitario → DecimalField

fecha → DateTimeField (auto_now_add)

Agregar propiedad calculada:

```
@property
def total(self):
    return self.cantidad * self.precio_unitario
```

Pasos:

1. Crear modelo en dashboard_app/models.py
2. makemigrations
3. migrate
4. Registrar en admin.py
5. Crear algunas ventas de prueba

Tiempo: 12 minutos

Queries útiles para el Dashboard

Queries de agregación:

```
from django.db.models import Sum, Count, Avg

# Total de ventas
Venta.objects.count()

# Suma de cantidades vendidas
Venta.objects.aggregate(total=Sum('cantidad'))

# Promedio de precio
Producto.objects.aggregate(avg=Avg('precio'))

# Agrupar ventas por producto
Venta.objects.values('producto__nombre')
  .annotate(total=Sum('cantidad'))
```

Filtros avanzados:

`__lte`	menor o igual
`__gte`	mayor o igual
`__contains`	contiene (LIKE)
`__startswith`	empieza con
`__year`	filtrar por año
`__month`	filtrar por mes
`__icontains`	contiene (sin mayúsc.)
`__in`	está en lista

Clase 32: Más sobre QuerySets y relaciones avanzadas

Resumen — Django Models

Lo que logramos hoy:

- Entendimos qué es un ORM
- Creamos modelo Categoría
- Creamos modelo Producto con FK
- Aplicamos migraciones
- Configuramos Admin personalizado
- Usamos el shell para queries
- Conectamos modelos con views
- Creamos modelo Venta

PRÓXIMA CLASE: Models II

- ForeignKey en profundidad
- ManyToManyField
- QuerySets avanzados
- Optimización de queries

Dashboard E-commerce:

Setup → URLs → Templates → Models ✓